

Assignment 4: The Archive Robot and the Corrupted Messages

Version 1.0.0 | English student specification

The archive robot has two damaged copies of a message, blocks that cost too much to merge, and a dictionary of phrases that should ring an alarm. It understands exact characters and algorithms. It does not understand why the printer asks for a reboot during every reboot. Help it recover shared content, coordinate a team and index the archive.

Learning goals

Implement, explain and validate Prefix/interval/tree DP, exact pattern matching, suffix ordering, Kasai LCP and failure-link automata. Recognise ties and edge cases; reconstruct valid answers and justify complexity.

Prerequisites

Arrays, recursion/postorder, trees, string indexing, modular arithmetic and sorting.

Work in small milestones

Shared subsequence -> interval plan -> tree team -> one-pattern search -> suffix index -> many-pattern automaton. Reuse the transport and tests, while keeping algorithm states independent.

Practical rubric (50 points)

Task	Algorithm	Requirement group	Points
1	lcs	core	4
1	lcs	witness	2
1	lcs	scale	2
2	merge	core	3
2	merge	witness	2
2	merge	scale	2
3	tree	core	3
3	tree	witness	2
3	tree	scale	2
4	naive	core	2
4	kmp	core	3
4	rk	core	3
4	kmp	scale	2
5	suffix	core	4
5	suffix	witness	3
5	suffix	scale	2
6	aho	core	4
6	aho	edge	3
6	aho	scale	2

core: optimum values/basic behaviour; edge: published edge cases; witness: feasible optimal reconstruction/condensation; scale: correct large-instance results. Each group has a fixed maximum. The sum is 50.

Task 1 - Recover Shared Message / LCS

Recover an ordered common message from two damaged copies; missing characters may be skipped.

Input: {task:lcs,a:string,b:string}

Output: {length,sequence:string}

Limits: $0 \leq |a|, |b| \leq 1000$; $|a| * |b| \leq 1000000$.

Implement prefix dynamic programming and reconstruct one longest common subsequence. sequence must be a subsequence of both copies and have the optimal length. It need not be contiguous. Any optimal sequence is accepted. Empty copies return length=0 and sequence="". Explain states, transitions, base cases and reconstruction in ANALYSIS.md.

Complexity: $O(|a| * |b|)$ time and memory are permitted.

Example input:

```
{"task": "lcs", "a": "tea_time", "b": "team"}
```

One accepted output:

```
{"length": 4, "sequence": "team"}
```

The final m is later in tea_time, so the recovered message is not a contiguous substring.

Task 2 - Merge Archive Blocks / Interval DP

Combine neighbouring archive blocks without repeatedly copying the biggest block first.

Input: {task:merge,sizes:[...]}

Output: {cost,plan:[[l,k,r],...]}

Limits: $0 \leq n \leq 200$; $0 \leq \text{size} \leq 1000000000$.

A current block represents a closed original interval. Merge adjacent blocks $[l,k]$ and $[k+1,r]$; pay their combined size and replace them by $[l,r]$. Return the minimum total cost and a chronological valid plan using original indices. Each input block begins as $[i,i]$. For $n \leq 1$ cost=0 and plan=[]. Any optimal plan is accepted. Greedily choosing the smallest two arbitrary blocks violates adjacency.

Complexity: $O(n^3)$ time, $O(n^2)$ memory; no advanced DP optimization is required.

Example input:

```
{"task": "merge", "sizes": [2, 3, 4]}
```

One accepted output:

```
{"cost": 14, "plan": [[0, 0, 1], [0, 1, 2]]}
```

First cost 5, then cost 9. The alternative costs $7+9=16$.

Task 3 - Recovery Team / Tree DP

Choose archive robots whose adjacent charging stations will not overload each other.

Input: {task:tree,n,weights:[...],edges:[[u,v],...]}

Output: {weight,selected:[vertex IDs,...]}

Limits: $0 \leq n \leq 100000$; $-1000000000 \leq \text{weight} \leq 1000000000$. For $n > 0$ input is a connected undirected tree with $n-1$ distinct edges; $n=0$ has no edges.

Implement maximum weight independent set by take/skip tree DP and reconstruct a selected set. The empty set is allowed, so all-negative inputs return weight 0. No adjacent vertices may both be selected. Any optimum is accepted. Root choice must not change the optimum; avoid recursion failure on deep chains.

Complexity: $O(n)$ time and memory.

Example input:

```
{"task": "tree", "n": 3, "weights": [4, 9, 6], "edges": [[0, 1], [1, 2]]}
```

One accepted output:

```
{"weight": 10, "selected": [0, 2]}
```

The two outer robots contribute 10, beating the central robot's 9.

Task 4 - Find a Phrase / Naive Search

Build a simple exact-search baseline for the archivist's favourite phrase.

Input: {task:naive,text:string,pattern:string}

Output: {positions:[...]}

Limits: $0 \leq |text| \leq 3000$; $0 \leq |pattern| \leq 3000$.

Implement naive character comparisons. Return ALL zero-based occurrence starts in strictly increasing order, including overlaps. Empty pattern matches every boundary $0..|text|$, including the final boundary. Pattern longer than text returns []. No built-in search functions in the algorithm.

Complexity: $O(n*m+n+1)$ time, $O(\text{number of answers})$ output memory.

Example input:

```
{"task": "naive", "text": "aaaaa", "pattern": "aaa"}
```

One accepted output:

```
{"positions": [0, 1, 2]}
```

Overlaps count: each alarm can start before the previous one has finished.

Task 4 - Find a Phrase / KMP

Reuse known prefix matches instead of asking the same character the same question again.

Input: {task:kmp,text:string,pattern:string}

Output: {positions:[...]}

Limits: $0 \leq |text| \leq 200000$; $0 \leq |pattern| \leq 100000$.

Implement prefix function and KMP. Use exactly the same matching contract as naive, including the empty pattern. Keep overlaps after a full match using prefix fallback. Compare outputs of all three search algorithms on inputs in their common domain.

Complexity: $O(n+m+\text{number of answers})$ time, $O(m+\text{number of answers})$ memory.

Example input:

```
{"task": "kmp", "text": "abababa", "pattern": "aba"}
```

One accepted output:

```
{"positions": [0, 2, 4]}
```

Resetting the matched-prefix length to zero after a match can miss valid overlaps.

Task 4 - Find a Phrase / Rabin-Karp

Use rolling fingerprints to reject unequal windows quickly, then verify each candidate.

Input: {task:rk,text:string,pattern:string}

Output: {positions:[...]}

Limits: $0 \leq |text|, |pattern| \leq 10000$; $|text| * |pattern| \leq 20000000$.

Implement polynomial rolling hash. Use base 31, modulus 101 and ASCII character codes; highest power belongs to the leftmost character. This deliberately small modulus makes collisions testable. Normalize negative remainders. Equal hashes are candidates only: compare actual characters before reporting. The exact matching contract, including empty patterns and overlaps, is shared with KMP. 64-bit arithmetic is sufficient. No built-in search functions.

Complexity: $O(n+m+c*m)$ time where c is the number of equal-hash windows; worst case $O(n*m)$.

$O(m+\text{answers})$ rolling-window memory, or $O(n+\text{answers})$ with prefix hashes, is permitted. Do not claim deterministic linear time.

Example input:

```
{"task": "rk", "text": "abdj", "pattern": "ab"}
```

One accepted output:

```
{"positions": [0]}
```

ab and dj both hash to 75 modulo 101 with base 31. Only ab is an actual match.

Task 5 - Repeated Messages / Suffix Array and LCP

Find the longest exact repeated fragment; the robot would like fewer duplicate 'please reboot' messages.

Input: {task:suffix,text:string}

Output: {sa:[...],lcp:[...],repeated:string}

Limits: $0 \leq |text| \leq 50000$. Build exactly the nonempty suffixes; do not include a sentinel or the empty suffix in output.

Implement prefix doubling and Kasai. sa is a permutation of $0..n-1$ in lexicographic suffix order. lcp has length n ; $lcp[0]=0$ and $lcp[i]$ compares $sa[i-1]$ with $sa[i]$. Return any longest substring occurring at two distinct starts; overlaps are allowed. If none exists, repeated="". Empty input has both arrays empty. Do not materialize all suffix strings.

Complexity: $O(n \log^2(n+1))$ time using comparison sorting of rank pairs is allowed; $O(n \log(n+1))$ with radix/counting sorting is also allowed. Kasai $O(n)$; $O(n)$ auxiliary memory.

Example input:

```
{"task": "suffix", "text": "banana"}
```

One accepted output:

```
{"sa": [5, 3, 1, 0, 4, 2], "lcp": [0, 1, 3, 0, 0, 2], "repeated": "ana"}
```

ana starts at positions 1 and 3; these occurrences overlap.

Task 6 - Alarm Dictionary / Aho-Corasick

Count every dictionary phrase across corrupted messages. 'tea' may be useful; 'reboot reboot' is usually less reassuring.

Input: {task:aho,patterns:[nonempty strings,...],texts:[strings,...]}

Output: {counts:[[count for each dictionary position],...]}

Limits: $0 \leq \text{number of patterns} \leq 10000$; total pattern length ≤ 100000 ; $0 \leq \text{number of texts} \leq 100$; total text length ≤ 200000 . Patterns are nonempty; texts may be empty.

Build one Aho-Corasick trie/automaton per request, with suffix/failure links. Query it for each independent text; reset state and counts. Count overlaps and suffix-pattern matches. Duplicate patterns are separate dictionary positions and receive equal counts. An empty dictionary produces an empty count row per text. An empty texts array produces []. Do not replace the automaton by multiple KMP searches. Count results using visit propagation along failure links or correct output-link traversal.

Complexity: Build $O(95 \cdot S)$ with fixed ASCII alphabet, or a justified sparse equivalent. Per-text counting $O(|\text{text}| + S + p)$ with failure propagation is allowed, or $O(|\text{text}| + \text{matches} + p)$ with output links. S is trie states and p is dictionary size. Avoid copying quadratically many inherited output lists.

Example input:

```
{"task": "aho", "patterns": ["he", "she", "he"], "texts": ["sheshe", "sheshe", ""]}
```

One accepted output:

```
{"counts": [[2, 2, 2], [2, 2, 2], [0, 0, 0]]}
```

he is a suffix of she; both count. Repeating the query does not accumulate earlier counts.

Common transport and reset rules

One JSON object per input line, one JSON result object per line. One task per request. No prompts or debug output on stdout. Independent requests reset all task state. Provided adapters handle input/output. Inputs satisfy the published domain; malformed JSON is not graded.

UTF-8; strings contain printable ASCII characters (32..126), case-sensitive; no Unicode normalization

All vertices, edges, characters and original blocks use zero-based indices. An edge ID is its position in the input edge array. All costs, distances, weights and counts use signed 64-bit integers; null denotes unreachable distance.

Assessment: exactly 50 + 50

Each assignment has practical work worth 50 and an individual in-class quiz worth 50. Their sum is the assignment score out of 100. Each assignment contributes 20% of the second attestation; the separate Quiz II and Endterm keep their syllabus weights of 20% and 40%. These new quizzes are part of the assignments, not replacements for Quiz II.

Practical points belong to published requirement groups. Within each group: $\text{group points} \times \frac{\text{passed checks}}{\text{total checks}}$, rounded to 0.01 with half-up rounding. The practical sum is capped at 50. Correct optimum values can earn core points even when a witness is invalid; reconstruction points require a valid optimal witness. Scale groups require correct output on published large input sizes within generous resource limits. A failed build prevents checks from earning points. A timeout loses the affected group checks. An internal checker error produces no grade and must be investigated and rerun.

Official checks use the submitted commit and a fixed version of the instructor checker, not your repository's workflow. Additional tests do not change a group's maximum points. Test data may be private, but all tested requirements are published here. Timing medians are for analysis; you are not ranked by milliseconds. The checker records functional correctness and scalability. Black-box tests alone cannot prove which algorithm you used; the stated implementation methods and individual understanding remain required.

Individual understanding quiz

20 questions, 2.5 points each, maximum 50; 30 minutes, one individual attempt in class without AI. Questions use small traces, states, invariants, counterexamples, complexity, reconstruction and changed conditions. Every attempt takes one question from each of 20 balanced categories. Four substantive variants per category are available. Single-choice and exact integer numerical answers use no negative marks or guessing penalty. An incorrect answer earns 0 for that question; ordinary automatic quiz marking has no partial credit. Dates are announced in Moodle; unpublished activities stay hidden. Practice quizzes A and B do not affect your grade.

Reconsidering specific quiz errors

You may ask to reconsider points lost on a particular question. This is optional, not a new compulsory defence. The instructor identifies the skill, gives a new equivalent example and asks you to explain the reasoning. Fully demonstrated understanding restores the points lost on that question; a correct main method with one significant unfinished part restores half of the lost points; otherwise the original score remains. Repeating a published answer without explanation is insufficient. The quiz maximum remains 50. Quiz reconsideration does not automatically change practical points. An erroneous question/key is corrected for all affected students.

Copy, run and submit

Open the official template link in Moodle and select Use this template -> Create a new repository. Choose any repository name. Public or private copies are allowed; if private, provide the instructor with access, or use the authorised local checking route. No payment or private repository is required. Do not fork an instructor checker or copy its private content.

Install your chosen runtime: Java JDK 17+, Python 3.12+, C++17 compiler, or .NET SDK 8. The launcher needs Python 3.12+ for all languages, matching Assignment 2. From the repository root run: `python tools/run.py --language Java --setup`. Replace Java with Python, Cpp or CSharp as appropriate. setup should pass before algorithm implementation. Run `python tools/run.py --language Java --check` after working on the algorithms; TODO methods intentionally fail their examples. Run a request file with `python tools/run.py --language Java --input examples/dsu.jsonl (A3) or examples/lcs.jsonl (A4)`. Windows may use `py -3` in place of `python`.

Implement the method bodies in your selected language's source folder. Keep JSON transport separate; it is provided. Add helper classes/functions in that source folder. No external algorithm libraries or built-in substring search may replace required implementations. Standard collections, sorting and priority queues are allowed. C++ targets Linux GCC for official checks; the provided header uses the GCC standard-library convenience header.

Complete ANALYSIS.md with 3-5 sentences per algorithm, three reproducible comparison input sizes, runtime/seed/commands and observations. Explain your states/invariants and reconstruction. This short report supports learning; it has no separate hidden marking group. Record AI tools and other resources used, their contribution, and what you verified. If none, say so. You must understand your submitted implementation; the quiz checks individual understanding. This project does not claim to prevent AI use.

Commit the finished source, your own tests, ANALYSIS.md and SUBMISSION.md. Set the selected language in submission.json. In Moodle submit your repository URL, full 40-character commit SHA and selected language. Record the SHA in the Moodle submission rather than making a self-referential commit. The official grade refers to that fixed commit, not a changing branch. Do not submit build binaries, credentials, private grader files or quiz keys. Dates and submission availability are provided in Moodle, not invented in this document.

Resource limits

Official compilation: 60 seconds. Each requirement group's published batch: 20 seconds, one CPU, 768 MiB container memory (Java heap up to 512 MiB), 128 processes, at most 8 MiB UTF-8 output. Large checks use up to 50000 graph/tree vertices, 100000 DSU operations, 60000 MST edges, 600x600 LCS states, 120 merge blocks, 100001 KMP text characters, 30000 suffix characters, and 600 alarm patterns over two 100000-character texts. These are representative scale checks inside the full published input domains. Small RK tests respect its honest worst-case bounds. Limits apply with headroom; output correctness and asymptotic analysis matter, not a tiny timing difference on shared CI.